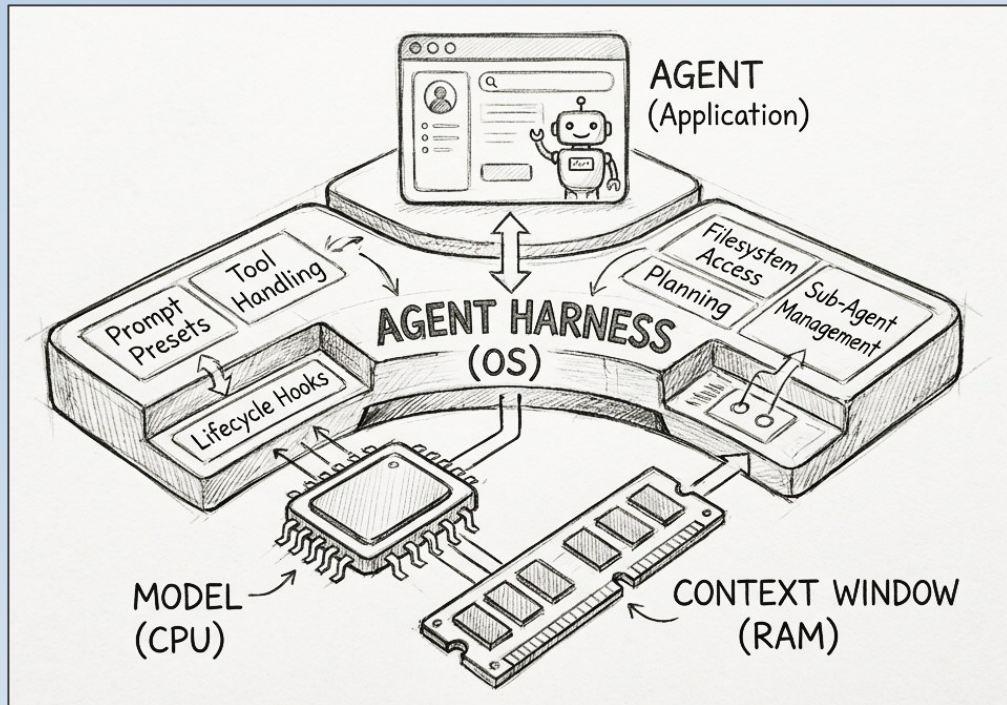


AIエージェント時代の ハーネスエンジニアリング Claude Code 実装例

国家資格キャリアコンサルタント
為安圭介

ハーネスエンジニアリングとは

AIエージェントの自律性と安全性を両立させるため、人間が望む方向に誘導する「手綱（ハーネス）となる環境」と「フィードバックループ」を設計する技術。



ハーネスエンジニアリングを構成する3つの柱



1. コンテキスト管理

AIに背景と目的、何をしたいのかを伝えつつ、特定のタスクに必要な仕様や現在の状態だけを精選して渡す仕組み



2. アーキテクチャ制約とガードレール

AIが「やってはいけないこと（依存関係のルール違反など）」を機械的にブロックする仕組み



3. 品質評価と多角的なフィードバック

人間ではなく、自動化されたテスト環境や評価用AI等を使って、AIの出力を継続的に検証・改善する仕組み

「エージェントハーネスは、AIモデルを包括し、長期実行タスクを管理するインフラストラクチャです。ハーネスは、プロンプトプリセット、ツール呼び出しの適切な処理、ライフサイクルフック、またはすぐに使用できる機能を提供します」

5層アーキテクチャで、AIの意図を実装に変える

Key Findings

- ① **ハーネス3柱を5層アーキテクチャで対応**
各層が制御の深度で積み上がる体系
- ② **各層でファイルが対応**
CLAUDE.md / skills/ settings.json / hooks / commands / agents まで
- ③ **実装すれば今日から動かせる**
5つの実装例を順番に試すだけでAIがチームになる



前回の3つの問いに、今回は5層の実装で答える

前回 概念編：エンジニアへの問い

問 1 AIに何を・どう渡すか？

(コンテキスト設計)

問 2 AIの暴走をどう防ぐか？

(制約・ガードレール)

問 3 品質をどう担保するか？

(評価・フィードバック)

→

今回 実装編：5層で答える

→

第1層 ルール層 (CLAUDE.md) でルールを固定

第2層 コンテキスト層 (MEMORY.md) で文脈を継続

→ スライド 5・6

→

第1層 ルール層 (settings.json) で権限制御

第3層 フック層 (hooks/) で実行前後をガード

→ スライド 7

→

第3層 フック層 (PostToolUse) で出力検証

第5層 エージェント・MCP層でレビュー担当を分離

→ スライド 10

So What? 3つの問いに、5層の実装ファイルで答える。概念を型に落とすことがエンジニアの仕事だ。

ハーネスはファイルで定義する

そのファイルは、対話で誰でも作れる

ハーネスエンジニアリングの成果物は"ファイル"だ

CLAUDE.md がAIのルールを定義する
hooks/*.py がAIの行動を制御する
skills/*.md がAIの仕事を型化する
agents/*.md がAIのチームを設計する

そのファイルは、Claudeとの対話で生まれる

エンジニアでなくても作れる
コードを書く必要はない
「何をしてほしいか」を伝えるだけでよい

対話でファイルが生まれる——実例3つ

CLAUDE.md

「/initでCLAUDE.mdの雛形を生成して。
そこに追記して：
ロールはキャリアコンサルタント、
出力は日本語、
git pushは承認なしに実行しない」

→ /init実行 → CLAUDE.md生成 → 追記完了

hooks

「Bashコマンドを実行する前に
rm -rf や --force が含まれていたら
実行をブロックするフックを作って。
/hooksで設定も更新して」

→ check_dangerous.py生成 + settings.json自動更新

skills

「note記事の執筆フローをスキルにしたい。
リサーチ→執筆→校正→投稿の順で、
各ステップで私の承認を入れて。
/draftで呼び出せるようにして」

→ .claude/skills/draft/SKILL.md生成

So What? "設計する人"になるのに、コーディング経験は不要だ。必要なのは"何をしてほしいか"を言語化する力だ

実装の全体像：5層アーキテクチャ

5層は "制御の深度" で積み上がる——上位層ほど人間の意図に近い

Layer	層名	役割	設置場所（対話で生成可能）
第1層	ルール層	AIの行動原則・ロール定義＋権限設定	CLAUDE.md / .claude/settings.json
第2層	コンテキスト層	セッション横断の記憶・エージェント別局所文脈	memory/MEMORY.md / agents/*.md
第3層	フック層	ツール実行前後のガードレール実装	settings.json > hooks / check_*.py
第4層	コマンド・スキル層	繰り返し作業の型化（基本:commands / 発展:skills）	.claude/commands/*.md / .claude/skills/*/SKILL.md
第5層	エージェント・MCP層	役割分担AIと外部ツール連携の統合管理	agents/*.md / settings.json > mcpServers

So What? まず第1層（ルール層）から始め、深度を上げながら実装していく。

実装例① 第1層 ルール層（CLAUDE.md + settings.json）

第1層 ルール層は "AIへの憲法" である——構造化すれば行動が安定する

※ CLAUDE.md は Claude Code との対話で生成可能（/init コマンドで雛形を一発生成）

Before — 構造なし

あなたはキャリアコンサルタントです。
日本語で教えてください。
丁寧に対応してください。

→ ロール・制約・禁止事項がバラバラ
→ セッションごとに挙動がブレる
→ 何を守れば良いかAIが判断できない

→

After — 構造化CLAUDE.md（第1層 ルール層）

ロール
1. Career Consultant — 個人の迷いを課題に再構成
2. Community Manager — 集団の知性へ変換する場を育てる

中核命題
体験を燃料に、言葉で輪郭を与え、関係と場の中で
学びと深い繋がりが続く状態をつくりつづける。

禁止事項
- 承認なしでファイル削除・git push を行わない

出力言語
日本語（コードのコメントも日本語）

作り方（第1層 ルール層）		
Step 1	Step 2	Step 3
/init を実行してCLAUDE.mdの雛形を生成	ルール・禁止事項・出力言語を対話で追記	「npmコマンドを許可して」と話しかけてsettings.jsonの権限設定を追加

構造化の4要素（第1層 ルール層の核心）

ロール定義	中核命題	禁止事項	出力言語
AIの「役割」を明示。複数ロールを番号付きで列挙する。	すべての行動の拠り所となる1文。行動基準の源泉。	やってはいけないことを明示することで「暴走」を防ぐ。	言語仕様を固定することで一貫性が生まれる。

So What? 最初を書く1ファイルが、AIの人格を決める。（第1層 ルール層）

実装例② 第2層 コンテキスト層（MEMORY.md + agents/*.md）

第2層 コンテキスト層は "3層で分離" する——全体・セッション・エージェントを分けることで爆発しない

※ コンテキスト＝Claudeが現在処理に使える情報の全体。コンテキストウィンドウが満杯になるとパフォーマンスが低下する（公式ドキュメントより）

コンテキスト分離（第2層 コンテキスト層の核心）			Before vs After	
第1層 CLAUDE.md 全体文脈 第2層 MEMORY.md セッション文脈 第2層 agents/*.md 局所文脈	全セッション共通の原則・ルール・禁止事項		Before 全情報一括渡し	After 第2層コンテキスト層で分離
	セッション横断の記憶・プロジェクト状態・途中経過		全ユーザー情報 + 全プロジェクト情報 + 全記事草稿 + 全メモ をそのまま渡す → トークン爆発でコスト増大 → 無関係情報でエラー・幻覚が増加	CLAUDE.md → 全エージェント共通原則のみ MEMORY.md → 今セッションの状態のみ agents/sage.md → Sage専用の役割のみ → 各層が最小限の情報だけ持つ → コスト削減 + 精度向上
	各エージェント専用の役割・制約（最小限）			

作り方（第2層 コンテキスト層）		
Step 1	Step 2	Step 3
CLAUDE.mdに「全体文脈」を記述（/initで雛形後に追記）	「プロジェクトの状態をMEMORY.mdに記録して」と話しかけてMEMORY.mdを生成	「〇〇エージェントの役割定義ファイルを作って」と話しかけてagents/*.mdを生成

生成指示例：「SageというリサーチエージェントのMDファイルを作って。担当はリサーチのみ」→ 以下を生成：

Sage — Research Agent
Role
Webリサーチ専門エージェント。検索・収集・要約のみ担当。
Context
- 全体原則はCLAUDE.mdを参照（第1層 ルール層）
- セッション状態はMEMORY.mdを参照（第2層）
- このファイルにはSage固有情報のみ記載

So What? AIに渡す文脈を設計すること＝第2層 コンテキスト層の実装だ。

実装例③ 第3層 フック層（PreToolUse / PostToolUse / Stop）

第3層 フック層は "AIのガードレール" である——実行の前後に検査を挟むことで暴走を防ぐ

※ フック（hook）＝ツール実行前後に自動で呼び出されるスクリプト。「～するフックを書いて」と話しかけるか、/hooksコマンドで設定できる



Before フックなし（第3層未実装）

AIが自由にツールを実行
→ 危険なコマンドも即時実行
→ git push --force が承認なしで走る
→ 本番ファイルが意図せず削除される

After 第3層 フック層 実装済み

PreToolUse でコマンドを事前検査
→ 危険パターン検出でブロック&警告
→ PostToolUse で出力の品質チェック
→ Stop フックで最終確認・ログ記録

作り方（第3層 フック層）

Step 1	Step 2	Step 3
「～するフックを作って」と話しかける 例：「危険コマンドをブロックするフックを作って」	ClaudeがPythonスクリプトを生成 + settings.jsonのhooks設定を自動更新	/hooks コマンドで設定内容を確認・調整

生成指示例：「Bashコマンド実行前にrm -rfや--forceをブロックするフックを作って」→ 以下を生成：

// settings.json (hooks設定)
"hooks": { "PreToolUse": [{
 "matcher": "Bash",
 "hooks": [{ "type": "command",
 "command": "python check_dangerous.py"}]}]
},
"PostToolUse": { "command": "python check_output_quality.py"}
},
"Stop": { "command": "python log_record.py"}
}

check_dangerous.py
import sys, json
data = json.load(sys.stdin)
cmd = data.get("command", "")
DANGER = ["--force", "rm -rf"]
for d in DANGER:

So What? 第3層 フック層は「AIの行動に制御を与える」ガードレールの実装だ。

実装例④ 第4層 コマンド・スキル層（commands + skills）

第4層 コマンド・スキル層は意図を型化する = Translation Bridgeの実装だ

※ 第4層 コマンド・スキル層 = 繰り返し作業を「型」にするファイル。「/commit」と打つだけで一定品質のコミットができる

比較軸	Commands（基本）	Skills（発展）
位置づけ	一般的なClaude Code機能	発展的な使い方（Commandsの上位版）
複雑度	シンプル・1ファイル完結	多ステップ・承認ルール付き
ファイル形式	.claude/commands/<name>.md	.claude/skills/<name>/SKILL.md
呼び出し方	/commit, /review など	/bizslide, /sage など
典型例	git commit の型化 定型レポート生成	スライド生成ワークフロー エージェント協調タスク

作り方（第4層 コマンド・スキル層）

Commands	Skills
「/commitというコマンドを作って。 動作は～」と話しかける → .claude/commands/commit.md生成	「～のワークフローをスキルにして。 各ステップに承認を入れて」と話しかける → .claude/skills/〇〇/SKILL.md生成

Before 毎回自然言語で指示

「前回と同じ形式でコミットして。メッセージは〇〇で。
Co-Authored-Byも忘れずに。emojiなしで。」
→ 毎回同じ指示を繰り返す / 品質がブレる



After /commit で一発（第4層）

- # .claude/commands/commit.md（対話で生成可能）
1. git diff --staged を確認
 2. Conventional Commits形式でメッセージ生成
 3. Co-Authored-By: Claude を付与 / emoji なし

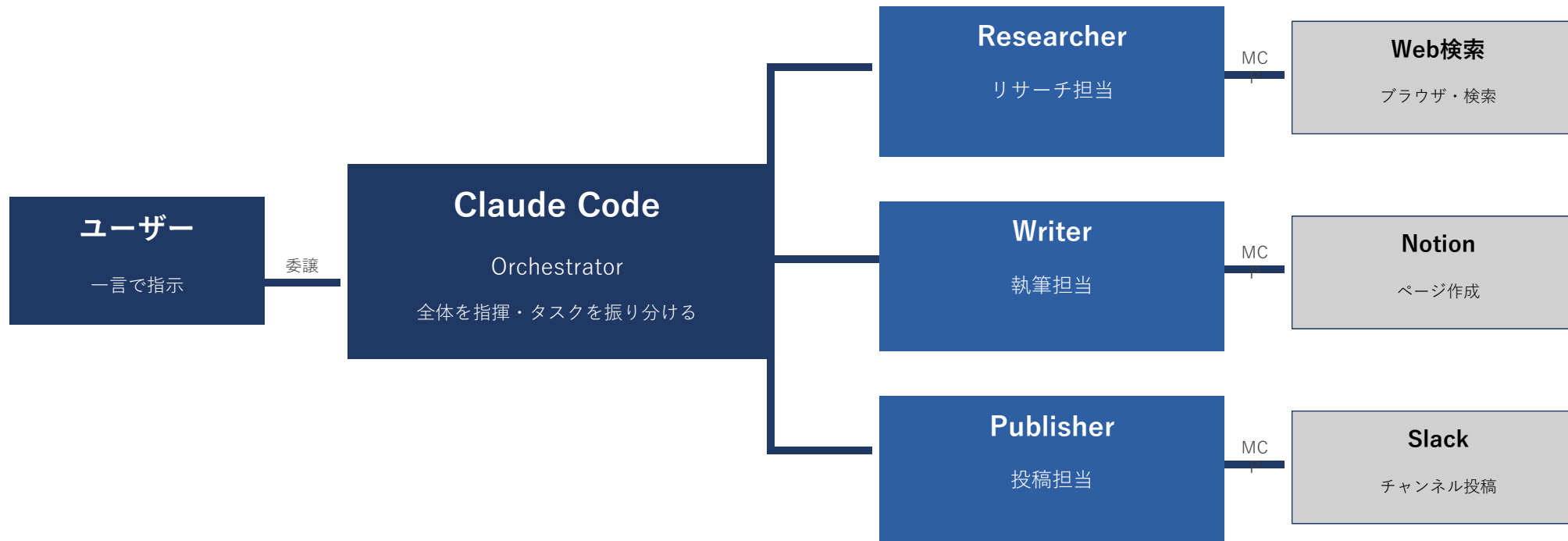
So What? 「型」があれば、誰でも同じ品質でタスクを実行できる。（第4層 コマンド・スキル層）

実装例⑤ 第5層 エージェント・MCP層 (agents + mcp-configs)

9

第5層 エージェント・MCP層を設計する = エージェントアーキテクトの仕事だ

※ 第5層 エージェント・MCP層 = 役割を与えられたAI個体 / MCP = Claudeが外部サービス进行操作するための標準プロトコル



エージェントの作り方

「〇〇という役割のエージェントを作って。担当は～のみ」と話しかける
→ .claude/agents/〇〇.md が生成される

MCPの接続方法

Step 1 : CLIコマンドで追加
claude mcp add
→ 対話形式でMCPサーバーを追加できる

Step 2 : 対話で追加
「GitHubのMCPサーバーを追加して。
リポジトリ読み書きとPR操作を許可して」
→ settings.jsonのmcpServersに自動追加

Step 3 : 設定結果 (settings.json)
"mcpServers": {
 "github": {
 "command": "npx",
 "args": ["-y", "@modelcontextprotocol/server-github"],
 "env": { "GITHUB_TOKEN": "~" }
 }
}

So What? 第5層 MCPを設定すれば、複数ツールをまたぐ作業がワンコマンドになる

品質評価は "ファイルを1つ書く" ことから始まる——そのファイルも対話で生成できる

※ フィードバックループ＝第3層フック（PostToolUse）で検査→問題あればブロック→Claudeが再試行、第5層エージェントが最終レビュー。check_output.pyも対話で生成できる。

フィードバックループ（第3層＋第5層）

1	AIが出力を生成
2	第3層 PostToolUse hook が実行 (check_output.py)
3	品質スコアが閾値以下なら自動リトライ
4	第5層 レビューエージェントが最終確認
5	結果を第2層 MEMORY.md に蓄積

リトライ

実装ファイル一覧（層対応）

層	ファイル	書くこと
第3層 出力検証	.claude/hooks/check_output.py	PostToolUseで品質チェック
第1層 リトライ制御	settings.json	hooks.PostToolUseにcheck_output.py登録
第5層 レビュー担当	.claude/agents/reviewer.md	検証専用エージェントの定義
第2層 学習蓄積	memory/feedback_log.md	検証結果・改善点の記録

作り方（品質評価ループ）

Step 1	Step 2	Step 3
「出力が空のときブロックするフックを作って」 → check_output.py生成	「出力を検証するレビューエージェントを作って」 → agents/reviewer.md生成	「検証結果をMEMORY.mdに記録して」 → 継続的な改善ループが完成

生成指示例（第3層 フック層）：「出力が空のときブロックし、禁止ワードを検出したら警告するフックを作って」 → 以下を生成：

```
# .claude/hooks/check_output.py
import json, sys
data = json.load(sys.stdin)
output = data.get("output", "")
if not output.strip():
    print(json.dumps({"decision": "block",
                     "reason": "空の出力を検出"}))
    sys.exit(0)
print(json.dumps({"decision": "approve"}))
```

So What? 第3層フック＋第5層エージェントで "フィードバックループも対話で作れる" ——実装の敷居は存在しない

5層を実装すれば、AIは "ツール" から "チーム" に変わる

①	第1層 ルール層：CLAUDE.mdを構造化する ロール・中核命題・禁止事項・出力言語の4要素で記述
②	第2層 コンテキスト層：3層に分離する CLAUDE.md / MEMORY.md / agents/*.md の役割を明確に
③	第3層 フック層：ガードレールを実装する PreToolUse + PostToolUse + check_*.py で検査
④	第4層 コマンド・スキル層：意図を型化する .claude/commands/ + .claude/skills/ にワークフローを蓄積
⑤	第5層 エージェント・MCP層：チームを設計する Sage / Lumina / Porter の役割分担 + MCP連携
⑥	第3層＋第5層：品質評価ループを自律化する PostToolUse + エージェントレビュー + 第2層MEMORY蓄積



前回3問いとの最終対応（5層表記）		
問1 コンテキスト設計 → 第1層 ルール＋第2層 コンテキスト（①②）	問2 制約・ガードレール → 第3層 フック（③）	問3 品質評価・フィードバック → 第3層 PostToolUse＋第5層 エージェントレビュー（⑥）

So What? ハーネスエンジニアリングは、5層で実装できる。



ためやす けいすけ
為安 圭介

NECソリューションイノベータ
ディレクター

NECソリューションイノベータにて、ITサービスによる
価値創造を軸に、事業と組織のマネジメントを担う。

AIコミュニティCDLEをはじめ、複数のコミュニティ運
営を行う傍ら、国家資格キャリアコンサルタントとして
情報発信にも注力し、学びと成長の場作りを実践。

活動拠点は北海道。

参加コミュニティ



趣味・好きなもの



取り組んでいること

- ・AIネイティブな組織づくり
- ・スキルアップの文化づくり
- ・CDLE歴史ブログ
- ・キャリア情報発信